DATABASE MANAGEMENT SYSTEMS – ASSIGNMENT CO2

Relational Design and Validation for Courier Handover Completion (SwiftSpan Parcels) – Brief Answers to Tasks 1–12

Assumptions used throughout

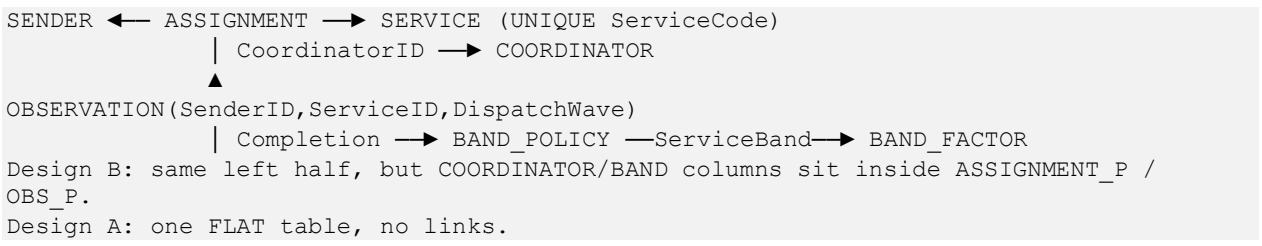
- 14 attributes: SenderID, SenderName, RouteGroup, ServiceID, ServiceCode, ServiceName, VolumeWeight, DispatchWave, CoordinatorID, CoordinatorName, Hub, Completion, ServiceBand, RouteFactor.
- Fictional policy: Completion 0–39 = Low (factor 0.8), 40–69 = Medium (1.0), 70–100 = High (1.2). Weighted adjusted index = Completion × VolumeWeight × RouteFactor.
- Seeded data (seed 42): 10 route groups, 50 catalog services (group g requires services 5g−4…5g), 40 coordinators, 8 hubs, 5 services × 4 waves per sender = 20 rows per actor (500 / 5,000 / 30,000 actors for 10k / 100k / 600k rows).
- **Measurements**in Tasks 4, 9, 10, 12 were taken locally with SQLite (Python, seeded, same data in every design; allocated bytes = page_count × page_size incl. PK indexes; median of 300 sender look-ups, warm cache). MySQL DDL in Task 8 is provided but not executed here. Estimates are labelled as such.

Task 1 – Alternative Relational Schemas (5)

Design	Relations (PK underlined by position first; * = UNIQUE)	Purpose
A. Flat baseline	FLAT(SenderID, SenderName, RouteGroup, ServiceID, ServiceCode*, ServiceName, VolumeWeight, DispatchWave, CoordinatorID, CoordinatorName, Hub, Completion, ServiceBand, RouteFactor). PK = (SenderID, ServiceID, DispatchWave); CK = (SenderID, ServiceCode, DispatchWave).	Repeated-data baseline; fastest single-table read.
B. Partial (2NF)	SENDER(SenderID, SenderName, RouteGroup); SERVICE(ServiceID, ServiceCode*, ServiceName, VolumeWeight); ASSIGNMENT_P(SenderID, ServiceID, CoordinatorID, CoordinatorName, Hub); OBS_P(SenderID, ServiceID, DispatchWave, Completion, ServiceBand, RouteFactor).	Removes sender/catalog repetition but keeps coordinator and policy redundancy.
C. BCNF	SENDER(SenderID, SenderName, RouteGroup); SERVICE(ServiceID, ServiceCode*, ServiceName, VolumeWeight); COORDINATOR(CoordinatorID, CoordinatorName, Hub); ASSIGNMENT(SenderID, ServiceID, CoordinatorID); BAND_POLICY(Completion, ServiceBand); BAND_FACTOR(ServiceBand, RouteFactor); OBSERVATION(SenderID, ServiceID, DispatchWave, Completion).	Every FD determinant is a key; one place per fact.

References: B: OBS_P(SenderID,ServiceID) → ASSIGNMENT_P; ASSIGNMENT_P → SENDER, SERVICE. C: ASSIGNMENT → SENDER, SERVICE, COORDINATOR; OBSERVATION(SenderID,ServiceID) → ASSIGNMENT; OBSERVATION.Completion → BAND_POLICY; BAND_POLICY.ServiceBand → BAND_FACTOR. ServiceCode is the alternate key of SERVICE (UNIQUE NOT NULL).

Schema diagram (determinant → dependent links, design C):



Why test all three: A shows the cost of repetition (55,000 look-ups need no join), B shows that removing only identity/catalog repetition is not enough, C is the correct schema for the update workload but needs 6 joins to rebuild one row.

Task 2 – Functional Dependencies (2)

#	FD	Business rule justifying it
F1	SenderID → SenderName, RouteGroup	A sender has one name and one declared group.
F2	ServiceID → ServiceCode, ServiceName, VolumeWeight	ServiceID identifies one catalog object.
F3	ServiceCode → ServiceID	ServiceCode is unique (alternate identifier).
F4	SenderID, ServiceID → CoordinatorID	One coordinator per sender/service pair across all waves.
F5	CoordinatorID → CoordinatorName, Hub	Coordinator master data.
F6	SenderID, ServiceID, DispatchWave → Completion	Observation key determines the 0–100 integer.
F7	Completion → ServiceBand	Fixed fictional value-to-band policy.
F8	ServiceBand → RouteFactor	Policy table of factors.

Classification against key {SenderID, ServiceID, DispatchWave}: Full: F6. Partial: F1, F2, F3, F4 (depend on a proper subset of the key). Transitive: CoordinatorName/Hub via CoordinatorID (F4→F5); ServiceBand via Completion (F6→F7); RouteFactor via ServiceBand (F7→F8).

Rejected apparent FDs: RouteGroup → CoordinatorID or ServiceID (generated cohorts happen to share values, but no rule forces it); DispatchWave → Completion or Hub (true only in one sampled wave); SenderName → SenderID (names are not declared unique). Only declared rules, not row patterns, create FDs.

Task 3 – Closures, Keys, Minimal Cover (6)

Closure of {SenderID, ServiceID, DispatchWave}: start = {S, Svc, W} → F1 adds SenderName, RouteGroup → F2 adds ServiceCode, ServiceName, VolumeWeight → F4 adds CoordinatorID → F5 adds CoordinatorName, Hub → F6 adds Completion → F7 adds ServiceBand → F8 adds RouteFactor. Result = all 14 attributes.

Closure of {SenderID, ServiceCode, DispatchWave}: F3 adds ServiceID, giving {S, Svc, W, ServiceCode}; then the same steps as above → all 14 attributes.

Armstrong proofs. (i) {SenderID,ServiceID} → CoordinatorName: F4 gives S,Svc→CoordinatorID; F5 gives CoordinatorID→CoordinatorName; transitivity gives S,Svc→CoordinatorName. (ii) {SenderID,ServiceCode,DispatchWave} → Completion: F3 + augmentation with S,W gives S,ServiceCode,W→S,Svc,W; F6 and transitivity give →Completion (decomposition/union give the rest of the row).

Candidate keys: K1 = {SenderID, ServiceID, DispatchWave}, K2 = {SenderID, ServiceCode, DispatchWave}. Proper subsets fail: {S,Svc} lacks DispatchWave (no FD gives Completion); {S,W} and {Svc,W} never reach CoordinatorID/Completion; {S}, {Svc}, {W} even less. Replacing Svc by ServiceCode is the only equivalence (F2/F3), so there are exactly two keys.

Prime attributes: SenderID, ServiceID, ServiceCode, DispatchWave. Non-prime: the other 10.

Minimal cover: remove the redundant FD {SenderID,ServiceID} → CoordinatorName (derivable via CoordinatorID) and the extraneous VolumeWeight in the augmented FD {ServiceID,VolumeWeight} → ServiceName (ServiceID alone gives ServiceName; the reduction is checked by ServiceID$^+$ ∋ ServiceName). The cover has 12 single-RHS FDs: S→SenderName; S→RouteGroup; Svc→ServiceCode; ServiceCode→Svc; Svc→ServiceName; Svc→VolumeWeight;

S,Svc→CoordinatorID; CoordinatorID→CoordinatorName; CoordinatorID→Hub; S,Svc,W→Completion; Completion→ServiceBand; ServiceBand→RouteFactor.

Equivalence: the cover ⊆ original trivially; each removed FD is in the closure of the cover (shown above), so the original ⊆ cover⁺. Hence they imply one another.

Task 4 – Anomalies (4)

- **Insertion:** in the flat design a new service (e.g. ServiceID 51, ServiceCode SC051) or a new coordinator cannot be stored until it has an observation (orphan catalog fact). In B/C they are inserted into SERVICE/COORDINATOR alone.
- **Deletion:** deleting the last observation of a service/coordinator in A (or of a coordinator's last assignment in B) erases ServiceName/CoordinatorName. Cause: ServiceID→ServiceName, CoordinatorID→CoordinatorName stored inside observation-level rows.
- **Update:** renaming ServiceID=1 must change every copy; a missed row leaves two ServiceNames for one ServiceID (violates F2). Rows touched (measured):

Update	10,000	100,000	600,000	Design
ServiceName, ServiceID=1	200 / 1 / 1	2,000 / 1 / 1	12,000 / 1 / 1	A / B / C
CoordinatorName, CoordinatorID=1	260 / 65 / 1	2,508 / 627 / 1	14,988 / 3,747 / 1	A / B / C
RouteFactor, band Low	3,840 / 3,840 / 1	39,462 / 39,462 / 1	237,547 / 237,547 / 1	A / B / C

Remaining repetition in B: CoordinatorName/Hub repeat per assignment (CoordinatorID→Name,Hub, transitive) and ServiceBand/RouteFactor repeat per observation (Completion→ServiceBand→RouteFactor). Storage (measured allocated MB incl. PK indexes; A / B / C): 10k: 0.99 / 0.59 / 0.52; 100k: 10.10 / 5.64 / 4.85; 600k: 61.16 / 34.09 / 29.41. Estimate: repeated text/number facts in A are ≈ 9 non-key columns × rows, versus ≈ 1 completion value per row in C.

Task 5 – Normal Forms up to BCNF (3)

Relation	Highest NF	Test using determinants
A. FLAT	1NF	Atomic values (1NF). F1–F4 are partial dependencies on a proper subset of the key, so 2NF fails.
B. ASSIGNMENT_P	2NF	Key (S,Svc); no partial FD. CoordinatorID→Name,Hub is transitive via a non-key determinant → 3NF fails.
B. OBS_P	2NF	Key (S,Svc,W); Completion→ServiceBand→RouteFactor transitive → not 3NF.
B. SENDER, SERVICE	BCNF	Determinants SenderID, ServiceID, ServiceCode are all keys.
C. all 7 relations	BCNF	Every determinant (SenderID, ServiceID/ServiceCode, CoordinatorID, (S,Svc), (S,Svc,W), Completion, ServiceBand) is a key of its own relation.

Synthesis (3NF from minimal cover, grouped by determinant): R1(S,SenderName,RouteGroup); R2(Svc,ServiceCode,ServiceName,VolumeWeight); R3(S,Svc,CoordinatorID); R4(CoordinatorID,Name,Hub); R5(S,Svc,W,Completion); R6(Completion,ServiceBand); R7(ServiceBand,RouteFactor); a key relation exists (R5 holds K1). Result equals Design C. Binary BCNF decomposition: split on S→{Name,Group}, then Svc→{…}, then (S,Svc)→CoordinatorID, CoordinatorID→{…}, Completion→ServiceBand, ServiceBand→RouteFactor; each split is on a determinant.

3NF without BCNF: R(A,B,D) with AB→D and D→B. Keys AB and AD; B is prime so 3NF holds, but D is not a key so BCNF fails. Decomposing into (D,B) and (A,D) is lossless but AB→D is no longer enforceable in one relation (not dependency-preserving) – the trade-off. SwiftSpan's own FDs have no such conflict, so BCNF is achievable with preservation.

Task 6 – Lossless Join, Dependency Preservation, 4NF/5NF (3)

Chase: tableau for the 7-way decomposition; applying F1, F2, F4, F5, F6, F7, F8 in order fills every row of the OBSERVATION-row with distinguished symbols (each step equates symbols using a determinant that is a key of one fragment), producing a fully distinguished row → lossless. Equivalent argument: each binary split R → R1(X^+) ∪ R2 has common attributes X as a key of R1.

Wrong split: OBS(S,Svc,W,Completion) split into (Svc,S) and (Svc,W,Completion) with common ServiceID: ServiceID is a key of neither, so the join pairs every sender of a service with every wave/value of that service – spurious tuples (lossy). The chase leaves non-distinguished symbols.

Dependency preservation: projecting F onto the 7 relations gives F1,F2,F3,F4,F5,F6,F7,F8 each inside one relation (F3 inside SERVICE), so every consequence stays enforceable by keys/FKs alone; Design C is both lossless and dependency-preserving.

4NF: SERVICE_MODE_DOC(ServiceID, TransportMode, DocType) with independent sets: ServiceID ↠ TransportMode | DocType (non-trivial MVD, ServiceID not a key) → BCNF but not 4NF. Decompose to SERVICE_MODE(ServiceID,TransportMode) and SERVICE_DOC(ServiceID,DocType).

5NF: AUTH(Operator, Item, Channel) where a triple is valid exactly when (Operator,Item), (Item,Channel) and (Channel,Operator) are all approved. This satisfies the join dependency ⋈{OI, IC, CO}, has no non-trivial MVD (so it is in 4NF) but violates 5NF; decompose into the three binary projections. Only apply if the three-way rule is a real business rule; otherwise keep the ternary table.

Task 7 – Relational Algebra and Calculus (4)

Notation: O = OBSERVATION, S = SENDER, V = SERVICE, A = ASSIGNMENT, C = COORDINATOR, BP = BAND_POLICY, BF = BAND_FACTOR; σ select, π project, ⋈ natural join, − difference, ÷ division; γ / ext = aggregate/extended operator (outside classical algebra).

Query	Formulation
Q1 Actor 1, wave 4, weighted index	RA: π_{ServiceID,Completion,Idx}(ext_{Idx = Completion×VolumeWeight×RouteFactor}(σ_{SenderID=1 ∧ DispatchWave=4}(O) ⋈ V ⋈ BP ⋈ BF)) [extended projection]. TRC: {t | ∃o∈O ∃v∈V ∃p∈BP ∃f∈BF (o.SenderID=1 ∧ o.DispatchWave=4 ∧ v.ServiceID=o.ServiceID ∧ p.Completion=o.Completion ∧ f.ServiceBand=p.ServiceBand ∧ t.ServiceID=o.ServiceID ∧ t.Completion=o.Completion ∧ t.Idx=o.Completion*v.VolumeWeight*f.RouteFactor)} [arithmetic extension]. DRC: {⟨sv,c,i⟩ | ∃w,b,r (⟨1,sv,4,c⟩∈O ∧ ⟨sv,w⟩∈V' ∧ ⟨c,b⟩∈BP ∧ ⟨b,r⟩∈BF ∧ i=c·w·r)} (V'(ServiceID,VolumeWeight)).
Q2 Senders meeting 39 for every required service	Good = π_{S,Svc}(A) − π_{S,Svc}(σ_{Completion<39}(O)); Req = π_{SenderID,ServiceID}(S ⋈ REQ(RouteGroup,ServiceID)). RA: π_{SenderID}(S) − π_{SenderID}(Req − Good). TRC: {s | ∃x∈S (s.SenderID=x.SenderID ∧ ∀r∈REQ (r.RouteGroup=x.RouteGroup → ∃a∈A (a.SenderID=x.SenderID ∧ a.ServiceID=r.ServiceID ∧ ¬∃o∈O (o.SenderID=a.SenderID ∧ o.ServiceID=a.ServiceID ∧ o.Completion<39))))}. DRC: {⟨s⟩ | ∃n,g ⟨s,n,g⟩∈S ∧ ∀v (⟨g,v⟩∈REQ → ∃k ⟨s,v,k⟩∈A ∧ ¬∃w,c(⟨s,v,w,c⟩∈O ∧ c<39))}.
Q3 Observed service records with no value below 39 in period 4	RA: π_{S,Svc}(σ_{W=4}(O)) − π_{S,Svc}(σ_{W=4 ∧ Completion<39}(O)) (first term avoids vacuous inclusion of unobserved items). TRC: {t | ∃o∈O (o.W=4 ∧ t.S=o.S ∧ t.Svc=o.Svc ∧ ¬∃o2∈O (o2.S=o.S ∧ o2.Svc=o.Svc ∧ o2.W=4 ∧ o2.Completion<39))}. DRC: {⟨s,v⟩ | ∃c ⟨s,v,4,c⟩∈O ∧ ¬∃c2(⟨s,v,4,c2⟩∈O ∧ c2<39)}.
Q4 Coordinator means below observation-weighted hub mean	RA (with γ extension): CM = γ_{CoordinatorID,Hub; AVG(Completion)→m}(O ⋈ A ⋈ C); HM = γ_{Hub; AVG(Completion)→hm}(O ⋈ A ⋈ C); answer = π_{CoordinatorID,m}(σ_{m<hm}(CM ⋈ HM)). TRC (aggregate extension): {t | ∃x∈CM ∃y∈HM (x.Hub=y.Hub ∧ x.m<y.hm ∧ t.CoordinatorID=x.CoordinatorID)}; AVG is not expressible in classical calculus.
Q5 Senders with sub-39 observations in ≥3 distinct waves	RA (γ): π_{SenderID}(σ_{n≥3}(γ_{SenderID; COUNT DISTINCT(W)→n}(σ_{Completion<39}(O)))). TRC (no aggregate): {s | ∃o1,o2,o3∈O (all SenderID=s ∧ all Completion<39 ∧ o1.W≠o2.W ∧ o2.W≠o3.W ∧ o1.W≠o3.W)}.

Note: Q1, Q2 (DRC shown), Q3 (DRC shown) satisfy the 'at least two in DRC' requirement.

Task 8 – SQL Implementation and Testing (4)

MySQL DDL (Design C, normalized):

```
CREATE TABLE sender(SenderID INT PRIMARY KEY, SenderName VARCHAR(60) NOT NULL,
RouteGroup TINYINT NOT NULL);
CREATE TABLE service(ServiceID INT PRIMARY KEY, ServiceCode VARCHAR(10) NOT NULL
UNIQUE,
  ServiceName VARCHAR(60) NOT NULL, VolumeWeight DECIMAL(4,2) NOT NULL);
CREATE TABLE coordinator(CoordinatorID INT PRIMARY KEY, CoordinatorName VARCHAR(60)
NOT NULL, Hub VARCHAR(20) NOT NULL);
CREATE TABLE band_factor(ServiceBand VARCHAR(10) PRIMARY KEY, RouteFactor DECIMAL(3,2)
NOT NULL);
CREATE TABLE band_policy(Completion INT PRIMARY KEY CHECK (Completion BETWEEN 0 AND
100),
  ServiceBand VARCHAR(10) NOT NULL, FOREIGN KEY(ServiceBand) REFERENCES
band_factor(ServiceBand) ON DELETE RESTRICT);
CREATE TABLE assignment(SenderID INT, ServiceID INT, CoordinatorID INT NOT NULL,
  PRIMARY KEY(SenderID,ServiceID),
  FOREIGN KEY(SenderID) REFERENCES sender(SenderID) ON DELETE RESTRICT,
  FOREIGN KEY(ServiceID) REFERENCES service(ServiceID) ON DELETE RESTRICT,
  FOREIGN KEY(CoordinatorID) REFERENCES coordinator(CoordinatorID) ON DELETE
RESTRICT);
CREATE TABLE observation(SenderID INT, ServiceID INT, DispatchWave TINYINT, Completion
INT NOT NULL,
  PRIMARY KEY(SenderID,ServiceID,DispatchWave),
  CHECK (Completion BETWEEN 0 AND 100),
  FOREIGN KEY(SenderID,ServiceID) REFERENCES assignment(SenderID,ServiceID) ON DELETE
RESTRICT,
  FOREIGN KEY(Completion) REFERENCES band_policy(Completion));
CREATE INDEX ix_obs_completion ON observation(Completion);
```

Designs A and B: FLAT and OBS_P/ASSIGNMENT_P are created with the same column types, PK(SenderID,ServiceID,DispatchWave) and the same CHECK, but without the FKs to coordinator/policy tables. All three load from the same seeded generator at 10k, 100k and 600k rows.

Five queries in SQL (design C):

```
-- Q1
SELECT o.ServiceID, o.Completion, o.Completion*v.VolumeWeight*f.RouteFactor AS idx
FROM observation o JOIN service v USING(ServiceID) JOIN band_policy p
USING(Completion)
JOIN band_factor f USING(ServiceBand) WHERE o.SenderID=1 AND o.DispatchWave=4;
-- Q2 (division by NOT EXISTS; REQ = services assigned within a RouteGroup)
SELECT s.SenderID FROM sender s WHERE NOT EXISTS (
 SELECT 1 FROM req r WHERE r.RouteGroup=s.RouteGroup AND (NOT EXISTS (SELECT 1 FROM
assignment a
   WHERE a.SenderID=s.SenderID AND a.ServiceID=r.ServiceID) OR EXISTS (SELECT 1 FROM
observation o
   WHERE o.SenderID=s.SenderID AND o.ServiceID=r.ServiceID AND o.Completion<39)));
-- Q3 (existence in wave 4 + anti-join)
SELECT o.SenderID,o.ServiceID FROM observation o WHERE o.DispatchWave=4 AND
o.Completion>=39;
-- Q4
WITH cm AS (SELECT c.CoordinatorID,c.Hub,AVG(o.Completion) m FROM observation o
  JOIN assignment a USING(SenderID,ServiceID) JOIN coordinator c USING(CoordinatorID)
GROUP BY 1,2),
 hm AS (SELECT c.Hub,AVG(o.Completion) hm FROM observation o JOIN assignment a
USING(SenderID,ServiceID)
  JOIN coordinator c USING(CoordinatorID) GROUP BY 1)
SELECT cm.CoordinatorID,cm.m FROM cm JOIN hm USING(Hub) WHERE cm.m<hm.hm;
-- Q5
SELECT SenderID FROM observation WHERE Completion<39 GROUP BY SenderID HAVING
COUNT(DISTINCT DispatchWave)>=3;
```

Views and privileges: v_actor_card (sender, service, coordinator, band columns, joined) and v_ops_summary (hub/coordinator aggregates, no sender names). Example (not executed): CREATE USER 'actor_app'@'%'; GRANT SELECT ON swift.v_actor_card TO 'actor_app'@'%'; REVOKE ALL ON swift.observation FROM 'actor_app'@'%'; GRANT SELECT ON swift.v_ops_summary TO 'ops_role'. A view limits columns, but it does not itself restrict which sender's rows a user sees unless the view filters on an identity (e.g. USER()) or the app enforces it.

Anomaly trials: START TRANSACTION; UPDATE service SET ServiceName='X' WHERE ServiceID=1; (and UPDATE coordinator SET CoordinatorName='Y' WHERE CoordinatorID=1;) check ROW_COUNT(); ROLLBACK; – run on A, B and C. Integrity tests: inserting Completion=101 (CHECK fails), an observation without assignment, or deleting a referenced service (restricted) must return an error.

Query sanity check (SQLite, local): the lookup query returned the same 20 rows per actor in all four structures at all sizes.

Task 9 – Experimental Results (3)

Local SQLite measurements (not MySQL; warm cache, in-process, so absolute times are tiny and only relative shape is meaningful). Lookup = all 20 rows of one sender with names, coordinator, band and factor.

Rows	Design	Allocated MB	Pages	Median lookup (ms)	Joins	Rows: ServiceName / CoordName / Factor
10,000	A flat	0.99	242	0.043	0	200 / 260 / 3,840
10,000	B partial	0.59	144	0.072	3	1 / 65 / 3,840
10,000	C BCNF	0.52	128	0.1	6	1 / 1 / 1
10,000	Read card	0.91	221	0.036	0	200 / n.m. / n.m.
100,000	A flat	10.10	2467	0.068	0	2,000 / 2,508 / 39,462
100,000	B partial	5.64	1376	0.059	3	1 / 627 / 39,462
100,000	C BCNF	4.85	1183	0.068	6	1 / 1 / 1
100,000	Read card	9.28	2265	0.041	0	2,000 / n.m. / n.m.
600,000	A flat	61.16	14932	0.044	0	12,000 / 14,988 / 237,547
600,000	B partial	34.09	8323	0.058	3	1 / 3,747 / 237,547
600,000	C BCNF	29.41	7179	0.084	6	1 / 1 / 1
600,000	Read card	56.37	13761	0.04	0	12,000 / n.m. / n.m.

n.m. = not measured. Integrity messages (expected in MySQL): CHECK violation on Completion=101, foreign-key failure for missing assignment/restricted delete. Equivalence: identical result rows per lookup across designs.

Task 10 – Theory vs Experiment (4)

- Updates match the determinants: ServiceID→ServiceName touches 12,000 rows in A at 600k (=600k/50) but 1 in B/C; CoordinatorID→Name touches 14,988 in A, 3,747 in B (once per assignment) and 1 in C; Completion→Band→Factor touches 237,547 rows in A and B but 1 in C. The retained redundancy in B is exactly the two transitive chains left in 2NF.
- Redundancy costs space: C is about 52% of A at 600k (29.4 vs 61.2 MB); B 34.1 MB sits between.
- Join cost: C needs 6 joins and was about 1.5–2× slower than A in median lookup (0.084 vs 0.044 ms at 600k); however, all are well below 1 ms here because the lookup is indexed by the PK prefix SenderID. This qualifies the claim that joins are 'expensive' – with correct keys they are cheap, but the ratio, not the absolute time, is what transfers.
- Index/cache effects: PK indexes make the look-up a range scan; a warm in-memory cache hides disk I/O, so results do not predict a production MySQL server (buffer pool, network, concurrency, MEMORY engine limits). No production guarantees are claimed.

Task 11 – Recommended Schema (4)

Recommendation: Design C (BCNF, 7 relations) as the source schema. It enforces every dependency F1–F8 with keys and foreign keys, is lossless and dependency-preserving, and is the only design where a policy, coordinator or service-name change is a single-row update (1 vs up to 237,547 rows measured). It stores 29.4 MB against 61.2 MB for A at 600k.

- Costs: more joins (6 for a full card), policy changes in BAND_POLICY/BAND_FACTOR must be re-validated against the 101 Completion values, and derived fields (ServiceBand, RouteFactor, index) are computed, not stored.
- Implemented and tested: PK/FK/UNIQUE/NOT NULL/CHECK/RESTRICT constraints and view columns. Unexecuted: the GRANT/REVOKE examples, which are illustrative only.
- Not chosen on the NF label alone: BCNF is selected because it also won on update cost and storage, with an acceptable join penalty.

Task 12 – Re-evaluate and Adapt the Design (8)

A. Impact analysis (2)

- New workload: 55,000 sender tracking look-ups per hour, with corrections committed every six hours (4 batches/day). Reads dominate; writes become batched and rare.
- 55,000 / 3,600 ≈ 15.3 requests/s on average (assume a 5× peak ≈ 76/s as a planning figure). Local single-query timing is not server throughput; throughput also depends on concurrency, network and connection handling.
- Earlier assumption of frequent row-level updates no longer dominates, so normalization's update advantage matters less per hour, but source integrity at each commit still matters.
- A controlled read projection (a read card) is acceptable if it is derived from the BCNF source, refreshed only by the batch, versioned/validated, and never edited directly – its repeated facts are then a cache, not a second source of truth.

B. Evaluation of original design (2)

600k rows	Joins	Median lookup (ms)	Allocated MB	Note
C normal (BCNF)	6	0.084	29.41	Authoritative source
Read card	0	0.040	56.37	≈ +27 MB (+92%) vs C

Measured: the card is about 2× faster locally (≈0.04 ms saved per look-up), but the local benchmark cannot establish hourly capacity, peak behaviour, lock contention or network effects; at ≈15 requests/s even C would be far below what such a lookup can serve, so the card is a headroom choice, not a necessity.
Options: ordinary view – always fresh, no extra storage, but still runs the 6 joins; transactional materialized read card (MySQL has no native MV, so a table rebuilt after each commit) – fast reads, bounded staleness (≤ 6 h), extra storage; permanently duplicated source table – fastest but re-introduces update anomalies (12,000+ rows per rename) and is rejected.

C. Decision: MODIFY (2)

- Retain the BCNF source schema (Design C). Add READ_CARD(SenderID, ServiceID, DispatchWave, SenderName, RouteGroup, ServiceName, VolumeWeight, CoordinatorName, Hub, Completion, ServiceBand, RouteFactor) with PK (SenderID,ServiceID,DispatchWave) and an index on SenderID, filled from the source.
- Consistency/refresh: after each six-hourly corrections commit, build READ_CARD_NEW in one snapshot (REPEATABLE READ) from the source, validate (row count = observation count, no NULLs, sample comparison with the join query), then RENAME TABLE swap atomically. Readers see either the old or new snapshot, never a mix.
- Measured consequences: extra 26.96 MB at 600k (10k: 0.38 MB; 100k: 4.43 MB); a ServiceName change is 1 row in the source but 12,000 rows in the card, paid once per refresh

rather than per edit; staleness up to 6 hours is acceptable for tracking but must be shown to users.

D. Evidence-based defence (2)

The source stays in BCNF with proven lossless join and preservation of all 8 FDs, so corrections are single-row and anomalies are impossible. Local results show the 6-join lookup is slightly slower than a join-free card (0.084 vs 0.040 ms), and the stated demand (≈15 requests/s average, ≈76/s assumed peak) is a planning input that justifies a cheap, rebuildable read structure rather than denormalizing the source. Six-hourly corrections make a batch refresh natural; the cost is ≈ 27 MB extra at 600k and ≤ 6 h staleness, both traded for faster reads. Redesigning to A would save the joins but cost 2× storage and 12,000–237,547-row updates, so it is rejected. These conclusions rest on local SQLite measurements and stated assumptions and should be re-verified on the target MySQL server under load.